



ktock simplify docs

6b8965e · 2 years ago



Name

Name

Last commit date



..



README.md

simplify docs

2 years ago

README.md



Running container on WASI runtimes with host networking

Networking on WASI runtimes is possible by relying on the network stack running on the host (outside of the WASI runtime).

We provide [c2w-net](#) as the user-space network stack running on the host. This is implemented based on [gvisor-tap-vsock](#).

The WASM image converted from the container can be configured to expose packets sent from the container via WASI's socket (`sock_*` API). WASI runtime binds the socket on a TCP port (e.g. using wasmtime's `--tcplisten` flag and wazero's [WithTCPListener](#) option). `c2w-net` running on the host connects to that port and forwards packets sent to/from the container.

The WASM image converted from the container has `--net` flag that enables this networking feature. `--net=socket` is the only supported mode which sends packets to `gvisor-tap-vsock` (wrapped by `c2w-net`) using [QEMU's forwarding protocol](#).

NOTE1: By default, the WASM image tries to establish connection with `c2w-net` via WASI's `fd=3`. However WASI runtimes might use larger `fd` when directory sharing is enabled. In that case, `--net=socket=listenfd=<num>` flag can be used for configuring the WASM image to use the correct socket `fd`.

NOTE2: This feature is tested only on Linux.

Example

This doc shows examples for wasmtime and wazero.

c2w and c2w-net are available on the [release page](#). You can also build them using make command:

\$ make



wasmtime

- Requirement
 - wasmtime needs to be installed.

`c2w-net` command with `--invoke` flag runs containers on wasmtime with automatically configuring it.

First, prepare a WASI image named `out.wasm`.

```
$ c2w alpine:3.18 /tmp/out/out.wasm
```



Then launch it on wasmtime with enabling networking.

```
$ c2w-net --invoke /tmp/out/out.wasm --net=socket sh
connecting to NW...
INFO[0001] new connection from 127.0.0.1:1234 to 127.0.0.1:50470
/ # apk update && apk add --no-progress figlet
apk update && apk add --no-progress figlet
apk update && apk add --no-progress figlet
fetch https://dl-cdn.alpinelinux.org/alpine/v3.18/main/x86_64/APKINDEX
fetch https://dl-cdn.alpinelinux.org/alpine/v3.18/community/x86_64/APKINDEX
v3.18.3-149-g8225da85c11 [https://dl-cdn.alpinelinux.org/alpine/v3.18/main/x86_64/APKINDEX/v3.18.3-149-g8225da85c11]
v3.18.3-151-g6953e6f988a [https://dl-cdn.alpinelinux.org/alpine/v3.18/community/x86_64/APKINDEX/v3.18.3-151-g6953e6f988a]
OK: 20071 distinct packages available
(1/1) Installing figlet (2.2.5-r3)
Executing busybox-1.36.0-r9.trigger
OK: 8 MiB in 16 packages
/ # figlet hello
figlet hello
figlet hello
```

[illegible]

|| | _ | | | _ /

It might takes several minutes to complete `apk add`.

Port mapping is also supported. The following example launches httpd server listening on `localhost:8000`.

[container2wasm](#) / [examples](#) / [networking](#) / **[wasi](#)** /

[↑ Top](#)

```
net=socket
```

It might takes several seconds to the server becoming up-and-running.

The server is accessible via `localhost:8000`.

```
$ curl localhost:8000
<html><body><h1>It works!</h1></body></html>
```



wazero

Wazero doesn't require `c2w-net` but the network stack can be directly implemented on the Go code that imports Wazero runtime. [../../../tests/wazero/](#) is an example command for wazero with enabling networking of the container. This is used in our integration test CI for wazero.

First, prepare a WASI image named `out.wasm`.

```
$ c2w alpine:3.18 /tmp/out/out.wasm
```



Then, it can run on wazero with networking support:

Run this at the project repo root directory.

```
$ ( cd ./tests/wazero && go build -o ../../out/wazero-test . )
$ ./out/wazero-test -net /tmp/out/out.wasm --net=socket sh
connecting to NW...
INFO[0001] new connection from 127.0.0.1:1234 to 127.0.0.1:53666
/ # apk update && apk add --no-progress figlet
apk update && apk add --no-progress figlet
fetch https://dl-
cdn.alpinelinux.org/alpine/v3.18/main/x86_64/APKINDEX.tar.gz
fetch https://dl-
cdn.alpinelinux.org/alpine/v3.18/community/x86_64/APKINDEX.tar.gz
v3.18.3-149-g8225da85c11 [https://dl-
```



```
cdn.alpinelinux.org/alpine/v3.18/main]
v3.18.3-151-g6953e6f988a [https://dl-
cdn.alpinelinux.org/alpine/v3.18/community]
OK: 20071 distinct packages available
(1/1) Installing figlet (2.2.5-r3)
Executing busybox-1.36.0-r9.trigger
OK: 8 MiB in 16 packages
/ # figlet hello
figlet hello

  _
 | | _   _ | | | _
 | ' _ \ / _ \ | | / _ \
 | | | | _/ | | ( ) |
 | _ | _ | \ _ | _ | \ _ /
```

It might takes several minutes to complete `apk add` .

Port mapping is also supported. The following example launches httpd server listening on `localhost:8000` .

```
$ c2w httpd /tmp/out/httpd.wasm
$ ./out/wazero-test -net -p localhost:8000:80 /tmp/out/httpd.wasm
--net=socket
```

It might takes several seconds to the server becoming up-and-running.

The server is accessible via `localhost:8000` .

```
$ curl localhost:8000
<html><body><h1>It works!</h1></body></html>
```